

Python

if __name__ == "__main__" to ensure code only runs when the script is executed directly, and not when it is imported as module.

In Python, everything is an object, and every object falls into one of two categories: mutable (changeable) or immutable (unchangeable). An immutable object cannot be altered after it is created. If you try to modify its value, Python doesn't actually change the original object; instead, it creates a brand new object in memory with the new value and points your variable to it. A mutable object can have its state or contents modified after it is created. You can add, remove, or change elements within the object, and it will remain the exact same object in memory.

Strings

```
message = 'Hello world'
print(message.lower())      # converts all characters to lower case
print(message.upper())     # converts all characters to upper case
print(message.count('l'))  # count number of passed argument
print(message.find('World')) # returns the starting index if string exist, else -1
new_message = message.replace('World', 'Universe') #str.replace returns a value
```

List, Tuples and Sets

```
courses = ['History', 'Math', 'Physics', 'CompSci']
print(len(courses))        #prints length of the list
courses.append('Art')      #add 'Arts' at the end of the list
courses.insert(0, 'Chemistry') #adds the element at the specified index
courses.extend(['Bio', 'PHE']) #adds the elements of list in argument to the object list

courses.remove('PHE')      #Remove first occurrence of the argument in the list,
                           #throws error if argument not present

popped = courses.pop()      #Removes the last element and returns it.
popped = courses.pop(0)    #removes the element at specified location, throws
                           #error if index out of range

courses.reverse()          #reverse the list
courses.sort(reverse = True) #sort the list, if reverse = True sort it in a
                           #descending order

sorted(courses)            #sorted function return sorted version of the list,
                           #without changing the list

nums = [101,45,56,87]
min(nums)                  #returns min of the list
max(nums)                  #returns max of the list
sum(nums)                  #returns sum of the list

Min, max return even on strings, whereas sum return error when list has strings.
courses.index('Math')      #returns first occurrence of the value, return error
                           #if not present

print('Hindi' in courses)  #check if value is present or not
#we want to join elements of courses by some identifier (here comma separated values)
newcourses = ', '.join(courses)
#Similarly, we want to split the string based on some identifier
newcourses.split(', ')
```

Tuples are immutable while list are mutable.

Set does not have duplicates and it does not care about an order.

```
mcourses = {'Physics', 'Chemistry', 'Math'}
bcourses = {'Physics', 'Chemistry', 'Bio'}
mcourses.intersection(bcourses) #Intersection between two sets
mcourses.union(bcourses)        #Union between two sets
mcourses.difference(bcourses)   #Difference between two sets
```

Initialization

```
empty_list = []
empty_list = list()
empty_tuple = ()
```

```
empty_tuple = tuple()
```

```
empty_set = set()
```

If we do `empty_set = {}`, it creates a dictionary, not a set

Dictionaries

```
student = {'name': 'John', 'age': 25, 'courses': ['Math', 'CompSci']}
```

Key-Value pair, if we access the key which does not exist, returns an error, if we don't want error, we can use `get()` method

```
print(student.get('name', 'Not_Found'))    #searches for the key, return Not_Found if not present
```

```
student['phone'] = '999999999'             #adding a new key-value pair
```

```
student.update({'name': 'Dan', 'age': 75}) #updating multiples values at a single time
```

#if we want to remove any key value-pair, we can use any of the following method

```
del student['Age']
```

```
age = student.pop('age')
```

```
students.keys()                            #return list of all keys
```

```
student.values()                           #return list of all values
```

```
student.items()                            #return list of all key-value pair in tuples
```

Falsy Values in Python (Values that evaluate to False):

- False
- None
- Zero of any numeric type 0, 0.0, 0j
- An empty sequence : '', (), []
- An empty mapping {}

When you create your own classes in Python, any object you create from that class will evaluate to True by default. It doesn't matter if the object has no attributes, empty lists inside it, or variables set to zero. To make your custom objects evaluate to False under specific conditions, Python looks for two specific dunder methods when it evaluates an object: `__bool__` and `__len__`. When you write `if my_custom_object:`, here is the exact order of operations Python follows behind the scenes:

- Does the object have a `__bool__()` method? If yes, execute it. Return its `True` or `False` result.
- If no, does it have a `__len__()` method? If yes, execute it. If it returns `0`, evaluate as `False`. Otherwise, `True`.
- If it has neither? Default to `True`.

Neither method defined (Defaults to True)

```
class DefaultObject:
    def __init__(self):
        self.empty_list = []
        self.zero_value = 0
```

Only `__len__` defined (0 is False, >0 is True)

```
class Box:
    def __init__(self, items):
        self.items = items
    def __len__(self):
        return len(self.items)
```

Both defined (`__bool__` takes priority)

```
class Account:
    def __init__(self, is_active, balance):
        self.is_active = is_active
        self.balance = balance
    def __bool__(self):
        return self.is_active
    def __len__(self):
        # This will be IGNORED for truthiness evaluation because __bool__ exists.
        return self.balance
```

```

default_obj = DefaultObject()
if default_obj:
    print("DefaultObject is True!")
else:
    print("DefaultObject is False!")

box1 = Box([])
box2 = Box(["apple", "orange"])
if not box1:
    print("Empty box")
else:
    print("Non Empty box")
if not box2:
    print("Empty box")
else:
    print("Non Empty box")

account1 = Account(is_active=True, balance=0)
account2 = Account(is_active=False, balance=1000)
if not account1:
    print("Inactive Account")# __bool__ overrides __len__
else:
    print("Active account")

```

***args and **kwargs** are powerful shortcuts that allow your functions to accept a variable (or arbitrary) number of arguments. A common misconception is that the words "args" and "kwargs" are reserved keywords in Python. They aren't. They are just widely accepted naming conventions established by the Python community. The actual magic happens because of the asterisks (* and **).

- * tells Python to pack extra positional arguments.
- ** tells Python to pack extra keyword arguments.

***args: The Tuple Maker :** When you use *args, Python takes all the extra arguments you pass in and packs them into a single tuple.

```

def multiply_everything(*args):
    result = 1
    for number in args:
        result *= number
    return result
print(multiply_everything(2, 3))           # Output: 6
print(multiply_everything(2, 3, 4, 5))     # Output: 120

```

****kwargs: The Dictionary Maker :** When you use **kwargs, Python takes all those named arguments and packs them into a dictionary.

```

def print_user_profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
print_user_profile(name="Alice", age=28, role="Admin")
# name: Alice
# age: 28
# role: Admin

```

